Shell Scripting

Table of Contents

1. Introduction
2. Basics
3. Variables
4. Input/Output
5. Operators
6. Conditional Statements
7. Loops
8. Functions

---

1. Introduction

What is a Shell?

```
Shell is a command-line interpreter that provides
interface between user and the OS kernel.

Types of Shells:

  sh   - Bourne Shell
  bash - Bourne Again Shell (most common)
  zsh  - Z Shell
  ksh  - Korn Shell
  csh  - C Shell
  fish - Friendly Interactive Shell
```

Check Your Shell

```
echo $SHELL          # Current default shell
cat /etc/shells      # All available shells
```

---

2. Basics

First Script

```
#!/bin/bash
# This is a comment
echo "Hello, World!"
```

Make It Executable

```
chmod +x script.sh      # Add execute permission
./script.sh             # Run the script
bash script.sh          # Or run with bash directly
```

Shebang Line Explained

```
#!/bin/bash             # Use bash
#!/bin/sh               # Use sh (POSIX)
#!/usr/bin/env bash     # Portable way (recommended)
#!/usr/bin/env python3  # Can use for other languages
```

Comments

```
# Single line comment

: '
This is a
multi-line
comment
'
```

```
<<'COMMENT'
Another way for
multi-line comments
COMMENT
```

---

3. Variables

Variable Declaration

```
#!/bin/bash

# No spaces around '=' sign!
name="John"             # String
age=25                  # Integer
PI=3.14                 # Float (treated as string)
readonly CONSTANT="fixed" # Read-only variable

echo $name
echo ${name}            # Recommended: with braces
echo "My name is ${name} and I'm ${age} years old"
```

Variable Types

```
#!/bin/bash

#       String Variables
first_name="John"
last_name="Doe"
full_name="${first_name} ${last_name}"

#       Integer Variables
declare -i num=10       # Explicitly integer
num=num+5               # Arithmetic without $(())

#       Read-only Variables
declare -r PI=3.14159
# PI=3.14     This would cause an error
```

```
#       Export Variables (Environment)
export PATH="$PATH:/new/path"
export MY_VAR="accessible in child processes"
```

Special Variables

```
#!/bin/bash

echo "Script Name     : $0"
echo "First Argument  : $1"
echo "Second Argument : $2"
echo "All Arguments   : $@"    # Each arg as separate word
echo "All Arguments   : $*"    # All args as single string
echo "Number of Args  : $#"
echo "Exit Status     : $?"    # 0=success, non-zero=error
echo "Process ID      : $$"
echo "Background PID  : $!"
echo "Last Argument   : ${!#}"
```

Variable Scope

```
#!/bin/bash

#       Global Variable (default)
global_var="I'm global"

my_function() {
    echo "$global_var"          # Accessible
    local local_var="I'm local" # Only inside function
    global_var="Modified"       # Modifies global
}

my_function
echo "$global_var"    # "Modified"
echo "$local_var"     # Empty - not accessible
```

Command Substitution

```
#!/bin/bash

# Modern syntax (preferred)
current_date=$(date)
file_count=$(ls | wc -l)
user=$(whoami)

# Old syntax (backticks)
current_date=`date`

echo "Date: $current_date"
echo "Files: $file_count"
echo "User: $user"

# Nested command substitution
files_in_home=$(ls $(echo $HOME) | wc -l)
```

---

4. Input/Output

Reading User Input

```
#!/bin/bash

#	Basic Input
echo "Enter your name:"
read name
echo "Hello, $name!"

#	Inline Prompt
read -p "Enter your age: " age

#	Silent Input (passwords)
read -sp "Enter password: " password
echo ""  # New line after silent input
```

```
#	Timeout
read -t 5 -p "Quick! Enter something (5 sec): " quick

#	Default Value
read -p "Enter color [blue]: " color
color=${color:-blue}

#	Read Multiple Values
read -p "Enter first and last name: " first last
echo "First: $first, Last: $last"

#	Read Into Array
read -a fruits -p "Enter fruits (space-separated): "
echo "First fruit: ${fruits[0]}"
```

Output Commands

```
#!/bin/bash

#	echo
echo "Normal text"
echo -n "No newline at end"
echo -e "Tab:\there\nNewline"   # Enable escape sequences
echo -e "\033[31mRed Text\033[0m"

#	printf (more control)
printf "Name: %s\n" "John"
printf "Age: %d\n" 25
printf "Price: %.2f\n" 19.99
printf "Hex: %x\n" 255
printf "%-20s %5d\n" "Alice" 90    # Formatted table
printf "%-20s %5d\n" "Bob" 85
```

Redirection

```
#!/bin/bash

#	Output Redirection
```

```
echo "Hello" > file.txt        # Overwrite
echo "World" >> file.txt       # Append
ls -la > listing.txt           # Save command output

#       Input Redirection
wc -l < file.txt               # Feed file as input
sort < unsorted.txt > sorted.txt

#       Error Redirection
ls /nonexistent 2> error.log           # Redirect stderr
ls /nonexistent 2>> error.log          # Append stderr
command 2>/dev/null                    # Suppress errors

#       Redirect Both stdout and stderr
command > output.txt 2>&1              # Both to file
command &> output.txt                  # Shorthand (bash)
command > output.txt 2> error.txt      # Separate files

#       Here Document
cat << EOF
Hello $USER
Today is $(date)
Your home is $HOME
EOF

#       Here Document (no variable expansion)
cat << 'EOF'
This $will not be expanded
EOF

#       Here String
grep "pattern" <<< "search in this string"
```

Pipes

```
#!/bin/bash

# Basic piping
ls -la | grep ".txt"
```

```
cat file.txt | sort | uniq
ps aux | grep nginx | grep -v grep

# Pipe to while loop
cat /etc/passwd | while IFS=: read user _ uid gid _ home shell; do
    echo "User: $user, UID: $uid, Shell: $shell"
done

# tee - write to file AND stdout
ls -la | tee output.txt
ls -la | tee -a output.txt    # Append mode
```

---

5. Operators

Arithmetic Operators

```
#!/bin/bash

a=20
b=5

#       Using $(( )) — Preferred
echo "Add:      $((a + b))"      # 25
echo "Subtract: $((a - b))"      # 15
echo "Multiply: $((a * b))"      # 100
echo "Divide:   $((a / b))"      # 4
echo "Modulus:  $((a % b))"      # 0
echo "Power:    $((a ** 2))"     # 400

#       Increment / Decrement
((a++))         # Post-increment
((a--))         # Post-decrement
((a += 5))      # Add and assign
((a -= 3))      # Subtract and assign
((a *= 2))      # Multiply and assign

#       Using expr (old way)
result=$(expr $a + $b)
```

```
result=$(expr $a \* $b)   # Must escape *

#      Using let
let "result = a + b"
let "a++"

#      Using bc for floating point
result=$(echo "scale=2; 10 / 3" | bc)     # 3.33
result=$(echo "scale=4; sqrt(2)" | bc -l)  # 1.4142
result=$(echo "3.5 + 2.1" | bc)           # 5.6
```

Comparison Operators

```
#!/bin/bash

#      Integer Comparison
# -eq  : Equal
# -ne  : Not equal
# -gt  : Greater than
# -ge  : Greater or equal
# -lt  : Less than
# -le  : Less or equal

a=10; b=20
[ $a -eq $b ]    # false
[ $a -lt $b ]    # true

#      Using (( )) for integer comparison
(( a == b ))     # Equal
(( a != b ))     # Not equal
(( a > b ))      # Greater than
(( a >= b ))     # Greater or equal
(( a < b ))      # Less than
(( a <= b ))     # Less or equal

#      String Comparison
str1="hello"
str2="world"
```

```
[ "$str1" = "$str2" ]     # Equal (POSIX)
[ "$str1" == "$str2" ]    # Equal (bash)
[ "$str1" != "$str2" ]    # Not equal
[ -z "$str1" ]            # Is empty?
[ -n "$str1" ]            # Is not empty?

#       Using [[ ]] (bash - preferred for strings)
[[ "$str1" == "$str2" ]]  # Equal
[[ "$str1" < "$str2" ]]   # Alphabetically less
[[ "$str1" > "$str2" ]]   # Alphabetically greater
[[ "$str1" == h* ]]       # Pattern matching
[[ "$str1" =~ ^hel ]]     # Regex matching
```

Logical Operators

```
#!/bin/bash

a=10; b=20; c=10

#       Using [ ]
[ $a -eq $c ] && [ $b -gt $a ]     # AND
[ $a -eq $b ] || [ $a -eq $c ]     # OR
[ ! $a -eq $b ]                    # NOT

#       Using [[ ]] (preferred)
[[ $a -eq $c && $b -gt $a ]]       # AND
[[ $a -eq $b || $a -eq $c ]]       # OR
[[ ! $a -eq $b ]]                  # NOT

#       Using (( )) for integers
(( a == c && b > a ))              # AND
(( a == b || a == c ))             # OR
```

File Test Operators

```
#!/bin/bash

file="test.txt"
```

```
dir="/tmp"

#	File Existence
[ -e "$file" ]	# Exists (file or directory)
[ -f "$file" ]	# Is a regular file
[ -d "$dir" ]	# Is a directory
[ -L "$file" ]	# Is a symbolic link

#	File Permissions
[ -r "$file" ]	# Is readable
[ -w "$file" ]	# Is writable
[ -x "$file" ]	# Is executable

#	File Properties
[ -s "$file" ]	# File size > 0
[ -O "$file" ]	# Owned by current user
[ -G "$file" ]	# Group matches current user

#	File Comparison
[ "$file1" -nt "$file2" ]	# Newer than
[ "$file1" -ot "$file2" ]	# Older than
[ "$file1" -ef "$file2" ]	# Same file (hard link)
```

---

6. Conditional Statements

if / elif / else

```
#!/bin/bash

#	Basic if
if [ $age -ge 18 ]; then
	echo "Adult"
fi

#	if-else
if [ $age -ge 18 ]; then
	echo "Adult"
else
```

```
    echo "Minor"
fi

#       if-elif-else
read -p "Enter score: " score

if [ $score -ge 90 ]; then
    grade="A"
elif [ $score -ge 80 ]; then
    grade="B"
elif [ $score -ge 70 ]; then
    grade="C"
elif [ $score -ge 60 ]; then
    grade="D"
else
    grade="F"
fi

echo "Grade: $grade"

#       One-liner
[ -f "file.txt" ] && echo "Exists" || echo "Not found"
```

Nested if

```
#!/bin/bash

read -p "Enter number: " num

if [ $num -gt 0 ]; then
    echo "Positive"
    if [ $((num % 2)) -eq 0 ]; then
        echo "Even"
    else
        echo "Odd"
    fi
elif [ $num -lt 0 ]; then
    echo "Negative"
else
```

```
    echo "Zero"
fi
```

case Statement

```
#!/bin/bash

read -p "Enter a fruit: " fruit

case $fruit in
    "apple"|"Apple")
        echo "    Red fruit"
        ;;
    "banana")
        echo "    Yellow fruit"
        ;;
    "grape"|"grapes")
        echo "    Purple fruit"
        ;;
    *)
        echo "Unknown fruit"
        ;;
esac

#       Advanced case with patterns
read -p "Enter filename: " filename

case $filename in
    *.txt)
        echo "Text file"
        ;;
    *.jpg|*.png|*.gif)
        echo "Image file"
        ;;
    *.sh)
        echo "Shell script"
        ;;
    [0-9]*)
        echo "Starts with a number"
```

```
        ;;
    *)
        echo "Unknown file type"
        ;;
esac
```

select Statement (Menu)

```
#!/bin/bash

echo "Choose your OS:"
select os in "Linux" "macOS" "Windows" "Quit"; do
    case $os in
        "Linux")
            echo "You chose Linux   "
            ;;
        "macOS")
            echo "You chose macOS   "
            ;;
        "Windows")
            echo "You chose Windows   "
            ;;
        "Quit")
            echo "Bye!"
            break
            ;;
        *)
            echo "Invalid option"
            ;;
    esac
done
```

---

7. Loops

for Loop

```
#!/bin/bash

#	Basic for loop
for name in Alice Bob Charlie; do
    echo "Hello, $name!"
done

#	C-style for loop
for ((i = 0; i < 10; i++)); do
    echo "Number: $i"
done

#	Range
for i in {1..10}; do
    echo "$i"
done

#	Range with step
for i in {0..100..5}; do
    echo "$i"
done

#	Loop over files
for file in *.txt; do
    echo "Processing: $file"
done

#	Loop over command output
for user in $(cat /etc/passwd | cut -d: -f1); do
    echo "User: $user"
done

#	Loop over array
fruits=("apple" "banana" "cherry")
for fruit in "${fruits[@]}"; do
    echo "Fruit: $fruit"
done

#	Loop with index
for i in "${!fruits[@]}"; do
```

```
    echo "Index $i: ${fruits[$i]}"
done
```

while Loop

```
#!/bin/bash

#       Basic while
count=1
while [ $count -le 5 ]; do
    echo "Count: $count"
    ((count++))
done

#       Infinite loop
while true; do
    read -p "Enter 'quit' to exit: " input
    [ "$input" = "quit" ] && break
done

#       Read file line by line
while IFS= read -r line; do
    echo "Line: $line"
done < "file.txt"

#       Read with counter
line_num=0
while IFS= read -r line; do
    ((line_num++))
    echo "${line_num}: $line"
done < "file.txt"

#       Read from command output
while IFS= read -r process; do
    echo "Process: $process"
done < <(ps aux)    # Process substitution
```

until Loop

```
#!/bin/bash

#	until loop (opposite of while)
count=1
until [ $count -gt 5 ]; do
    echo "Count: $count"
    ((count++))
done

#	Wait for a file to exist
until [ -f "/tmp/signal.txt" ]; do
    echo "Waiting for file..."
    sleep 1
done
echo "File found!"
```

Loop Control

```
#!/bin/bash

#	break
for i in {1..10}; do
    if [ $i -eq 5 ]; then
        break          # Exit loop
    fi
    echo "$i"
done
# Output: 1 2 3 4

#	continue
for i in {1..10}; do
    if [ $((i % 2)) -eq 0 ]; then
        continue       # Skip to next iteration
    fi
    echo "$i"
done
# Output: 1 3 5 7 9

#	break n (break out of n levels)
```

```
for i in {1..3}; do
    for j in {1..3}; do
        if [ $j -eq 2 ]; then
            break 2    # Break out of both loops
        fi
        echo "$i-$j"
    done
done
```

---

8. Functions

Basic Functions

```
#!/bin/bash

#       Method 1
greet() {
    echo "Hello, World!"
}

#       Method 2
function greet2 {
    echo "Hello again!"
}

# Call functions
greet
greet2
```

Functions with Parameters

```
#!/bin/bash

greet() {
    local name=$1       # First parameter
    local greeting=$2   # Second parameter
    echo "${greeting:-Hello}, $name!"
```

```
}

greet "Alice" "Hi"      # "Hi, Alice!"
greet "Bob"             # "Hello, Bob!"

#       Function with all parameters
show_args() {
    echo "Number of args: $#"
    echo "All args: $@"

    local count=1
    for arg in "$@"; do
        echo "  Arg $count: $arg"
        ((count++))
    done
}

show_args "one" "two" "three"
```